

Prepare the Pixel. Measure the handoff.

Use this guide with the technical helper and an ordinary observer. It covers the approved Google Pixel 8a, the disposable debug APK, and the workstation timing script. It does not authorize a Gate 0A pass.

Safety boundary — stop before changing the device

Do not flash an ISO or custom ROM, root, reformat, factory-reset, provision, or otherwise modify the Pixel except through the device owner’s already-approved managed-device procedure.

Do not enable or use live SMS, XMPP, carrier messaging, network messaging, location, camera, microphone, evidence capture, or production covert behavior.

The prepared package is local-only. It records device-local observations and launches the selected app’s normal launcher intent.

01 Record the run

DATE / UTC START	RUN NAME OR ID
OPERATOR	OBSERVER
REQUIRED DEVICE Google Pixel 8a	REQUIRED PLATFORM Stock Android · API 35

02 Physical-run no-go conditions

- ☐ The device is not the approved managed Google Pixel 8a, stock Android, API 35.
- ☐ ADB shows no device, more than one device, or a target marked **unauthorized** or **offline**.
- ☐ Only an emulator is available. Emulator output is not physical validation.
- ☐ The device owner cannot confirm the approved managed-device or device-owner procedure.
- ☐ The cover app is missing or has no normal launcher entry.

Stop / NO-GO: disconnect from the run, write the reason in the notes, and contact the technical helper or device owner. Do not repair a wrong device by resetting or modifying it. Do not infer a result from a screenshot, emulator, or host timing log.

03 Laptop and ADB preflight

Have the technical helper run the version and SDK checks in the laptop's PowerShell or terminal. Use PowerShell for the build/install commands and Git Bash, WSL, or another approved Bash environment for measure-gate0a.sh. The Android package has no checked-in Gradle wrapper; use the workstation's installed toolchain.

Expected workstation prerequisites

- ☐ JDK 17 is active. The Android build uses Java 17 source and target compatibility.
- ☐ Android SDK platform **android-35** and Android build-tools are installed.
- ☐ Gradle is installed and on PATH; the package intentionally uses the system gradle command.
- ☐ ADB is installed and on PATH. Enable USB debugging only under the approved device procedure.

VERSION AND SDK CHECKS

```
java -version
gradle --version
adb version

# PowerShell: use the configured SDK root, or replace $Sdk with its path.
$Sdk = if ($env:ANDROID_SDK_ROOT) { $env:ANDROID_SDK_ROOT } else { $env:ANDROID_HOME }
Test-Path "$Sdkplatformsandroid-35android.jar"
Get-ChildItem "$Sdkbuild-tools" | Sort-Object Name | Select-Object -Last 1
```

The SDK test must return **True** and the build-tools directory must contain an installed version. If the commands fail, stop and fix the workstation before touching the Pixel.

04 Authorize exactly one target

```
adb devices -l
adb shell getprop ro.product.model
adb shell getprop ro.build.version.sdk
adb shell getprop ro.build.version.release
```

In `adb devices -l`, there must be exactly one target row ending in **device**. On the phone, accept the USB-debugging RSA prompt only when the helper confirms this is the approved Pixel. The model check must read **Pixel 8a**; the SDK check must read **35**.

Do not continue on “unauthorized”, “offline”, a second target, or a different model/API. Stop and contact the device owner. Never use `adb -s` to hide an extra target or bypass the no-go decision.

05 Build and install the disposable APK

There are two prepared executables. Do not confuse them:

EXECUTABLE A · PIXEL

Disposable debug APK

Built on the laptop, then installed on the Pixel. The file is `app/build/outputs/apk/debug/app-debug.apk`. It provides the **CAS Pixel Gate 0A** screen, proxy, local journal, and report copy action.

EXECUTABLE B · WORKSTATION

measure-gate0a.sh

Run from the laptop after installation. It starts the local proxy through ADB and writes a host timing log. It is not installed on the Pixel and is not a physical result by itself.

Build and install from the package directory

```
cd artifacts/covert-alert-system/android-test-package
gradle :app:assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Confirm the build succeeds before installing. The install command targets the one authorized device from preflight and uses the exact prepared APK filename. Do not substitute a release APK, emulator APK, or another package.

06 Open the test package and set the cover

1. On the Pixel, open the normal launcher and start **CAS Pixel Gate 0A**.
2. Enter the exact package name of a harmless app already installed on the Pixel, then press **Save cover app**.
3. Verify that the selected app has a normal launcher entry and that it is the app the observer expects to see.
4. If the package is blank, missing, or has no normal launcher intent, stop and record **NO-GO**. The proxy records `NOT_LAUNCHED` and does not take another action.

COVER APP / EXACT PACKAGE NAME

07 Request and confirm the launcher shortcut

1. In **CAS Pixel Gate 0A**, press **Request pinned proxy shortcut**.
2. Accept the request in the Pixel launcher if the launcher asks for confirmation.
3. Return to the home screen and visually confirm the **Test cover launch** shortcut is actually pinned.
4. Only after the shortcut is visible, press it once and confirm that the selected cover app opens normally.

The app can only request a pin. The launcher owns the final result. A returned request, button press, in-app REQUESTED event, or screenshot of the request is not proof that the launcher pinned it. If the shortcut is absent, stop / NO-GO.

08 Prepare the supported measurements

Keep the technical helper beside the laptop. The observer watches the Pixel and writes what actually happens. The helper must explicitly supervise the script's Enter prompt, power-key lock/unlock steps, and reboot. Do not run the reboot step unattended.

DEVICE-SIDE CHECKS

- ☐ Cold / first launch
- ☐ Warm launch
- ☐ Locked-screen launch
- ☐ Post-reboot launch

OBSERVER CHECKS

- ☐ Back, Home, and Recents
- ☐ Settings → App info
- ☐ Notifications and Quick Settings
- ☐ Extra splash, frame, wrong task, or dead end

09 Run the host timing script

From the package directory on the workstation, give the log a run-specific name. The script waits for ADB, verifies the package, launches the local proxy with `am start -W`, and records host-side start/end/elapsed markers.

```
cd artifacts/covert-alert-system/android-test-package
bash scripts/measure-gate0a.sh gate0a-<RUN-ID>.log
```

What the helper must supervise

1. The script runs **cold-or-first** and **warm** samples.
2. When it prints “Unlock the device if needed, then press Enter for locked-screen sample,” make the agreed device state ready and press Enter only with the helper present.
3. The script sends a power key event, runs the **locked** sample, then sends another power key event.
4. The script prints “Rebooting device for the post-reboot sample,” runs `adb reboot`, waits for `sys.boot_completed=1`, and runs **after-reboot**. The helper must supervise this reboot and wait.
5. At the end, confirm the host log is saved as `gate0a-<RUN-ID>.log` in the package directory, unless an explicit path was supplied.

Important: this script does not perform an automated 200-repeat run. Any repeat-launch sample required by the handoff is a supervised physical repetition using the pinned shortcut and the approved repeat procedure; record the count and first failure in the notes. A host log, emulator run, or local timing value never substitutes for physical observation.

10 Record the five launch rows

A Cold / first	Agreed ready state; press the confirmed pinned shortcut once.	Result / time: _____
B Warm	Without rebooting, press the pinned shortcut again.	Result / time: _____
C Locked	Follow the helper’s lock-screen instruction; observe the transition.	Result / time: _____
D Post-reboot	After the helper confirms boot completed, press the shortcut.	Result / time: _____
E Repeats	Use the approved supervised repeat procedure; record count and first failure.	Count / result: _____

11 Copy the report and preserve the handoff

- 1. On the Pixel's **CAS Pixel Gate 0A** screen, press **Copy JSON report**. The app copies the current device-local report to the Android clipboard.
- 2. Using the approved workstation handoff, paste the clipboard text into gate0a-<RUN-ID>-report.json. Keep the file as JSON; do not edit or trim its contents.
- 3. Keep gate0a-<RUN-ID>.log from the workstation beside the JSON. Preserve the printed guide, repeat count, observer notes, and any NO-GO reason together.
- 4. In the CAS console, open **Feasibility gates**. In **Import Gate 0A report**, choose the JSON file and press **Validate & import report**.

Report contract to check before handoff

The report copied from the app is cas-gate0a-report-v1. It contains the target (Pixel 8a, API 35, stock Android), local-only safety flags, cover package, device-owner report, permissions, shortcut state, tasks, Recents, Back, observer review flags, and device-local events with wallClockMs and elapsedRealtimeMs.

Import is not a pass

CAS maps an accepted report to the **Proxy Launch** physical observation as **INCONCLUSIVE**. It preserves the raw timestamp notes for human review. Importing never creates Pass or GO; the final decision must use the physical screen observations, observer notes, and any failures.

12 Observer review and final record

- ☐ Cover app looked normal.
- ☐ Back and Home behaved normally.
- ☐ Recents had no stray proxy card.
- ☐ Settings → App info looked expected.
- ☐ Notifications looked expected.
- ☐ Quick Settings looked expected.
- ☐ No extra splash/frame or wrong task appeared.
- ☐ No dead end or unexpected transition appeared.

OBSERVER NOTES / FAILURES / REPEAT COUNT

RECORD ONE	REASON OR NEXT ACTION
☐ GO — only after human review of all required physical observations	
☐ NO-GO — blocked, wrong device, unsafe state, or failed check	
☐ INCONCLUSIVE — physical evidence or review is incomplete	